Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №3

з дисципліни
«Проєктування складних систем»

на тему

«Дослідження наслідків роботи технології апаратної реалізація

транзакційної пам'яті»

Виконав:

студент групи ІМ-21

Сірик Максим

Варіант 20

Перевірив:

Долголенко Олександр

Київ 2026

Мета роботи: Дослідити наслідки роботи технології апаратної реалізації транзакційної пам'яті

Постановка завдання

Для виконання цієї роботи потрібен доступ до мікропроцесора Intel, що підтримує технологію TSX-NI, або TSXLDTRK. Розгорніть на ньому якусь СУБД. Організуйте доступ до СУБД за допомогою спеціального API, наприклад, javax.sql чи java.sql. Розробіть тестову багатопоточну програму на Java для роботи з СУБД і дослідіть продуктивність своєї СУБД при використанні hardware transactional memory. Після цього відключіть на своєму мікропроцесорі підримку технології TSX-NI (див. виконання другої лабораторної роботи), підключіть якусь із бібліотек Java, що підтримує software transactional memory та дослідіть продуктивність цього варіанту СУБД на тій же тестовій програмі при використанні software transactional memory. Приведіть порівняльні графіки й зробіть висновки по роботі.

Виконання завдання

Лістинг

```
package com.github.erotourtes.structured.concurency;

import org.multiverse.api.references.TxnInteger;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.*;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Random;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

import static org.multiverse.api.StmUtils.*;

enum ExecutionMode {
    PLAIN,
    LOCK,
    STM
}

enum TransferExecutionOutcome {
    SUCCESS,
    INSUFFICIENT_FUNDS
}

interface CoordinationStrategy {
    void beforeTransfer(int fromAccount, int toAccount);

    void afterTransfer(int fromAccount, int toAccount);
}

record WorkerMetrics(
        long successfulTransfers,
        long insufficientFunds,
        long sqlErrors,
        long totalLatencyNanos
) {
    public WorkerMetrics add(WorkerMetrics other) {
```

```
        return new WorkerMetrics(
                successfulTransfers + other.successfulTransfers,
                insufficientFunds + other.insufficientFunds,
                sqlErrors + other.sqlErrors,
                totalLatencyNanos + other.totalLatencyNanos
        );
    }
}

record TransferOperation(int fromAccount, int toAccount,
                         int amount) {
}

record BenchmarkConfig(
        String jdbcUrl,
        int[] threads,
        ExecutionMode[] modes,
        /**
         * Total number of transfer operations to perform in each
benchmark run
         */
        int accountCount,
        /**
         * Number of "hot" accounts that are more likely to be involved
in transfers, creating contention.
         * Must be less than or equal to accountCount.
         */
        int hotAccountCount,
        int operationsPerRun,
        int initialBalance,
        int maxTransferAmount,
        int repeats,
        long seed,
        Path csvOutput
) {
    public static BenchmarkConfig defaultConfig() {
        return new BenchmarkConfig(
                "jdbc:h2:mem:lab3;DB_CLOSE_DELAY=-1;LOCK_TIMEOUT=10000",
                new int[]{1, 2, 4, 8},
                ExecutionMode.values(),
                1_000,
                20,
                50_000,
                10_000,
                250,
                3,
```

```
                42L,
                Path.of("build", "reports", "lab3-results.csv")
        );
    }
}

record BenchmarkResult(
        ExecutionMode mode,
        int threads,
        int repeat,
        long elapsedMillis,
        long successfulTransfers,
        long insufficientFunds,
        long sqlErrors,
        double throughputOpsPerSecond,
        double averageLatencyMicros
) {
}

public final class Main {
    public static void main(String[] args) throws Exception {
        BenchmarkConfig config = BenchmarkConfig.defaultConfig();
        Database database = new Database(config.jdbcUrl());
        BenchmarkRunner runner = new BenchmarkRunner(database);

        List<BenchmarkResult> results = runner.run(config);
        ResultPrinter.print(results);

        writeCsv(config.csvOutput(), results);
    }

    private static void writeCsv(Path output, List<BenchmarkResult>
results) throws IOException {
        CsvResultWriter.write(output, results);
        System.out.println("CSV written to " + output.toAbsolutePath());
    }

    public static final class WorkloadGenerator {
        private WorkloadGenerator() {
        }

        public static List<TransferOperation> generate(BenchmarkConfig
config) {
            Random random = new Random(config.seed());
            List<TransferOperation> operations = new
ArrayList<>(config.operationsPerRun());
```

```
            for (int i = 0; i < config.operationsPerRun(); i++) {
                /**
                 * 80% to hit the hot accounts
                 *
                 * Hot accounts     [1..hotAccountCount]
                 * Cold accounts    [hotAccountCount+1..accountCount]
                 */
                boolean hotOperation = random.nextDouble() < 0.8;
                int lowerBound = hotOperation ? 1 :
config.hotAccountCount() + 1;
                int upperBound = hotOperation ? config.hotAccountCount()
: config.accountCount();

                {
                    var tmp = upperBound;
                    upperBound = Math.max(tmp, lowerBound);
                    lowerBound = Math.min(tmp, lowerBound);
                }

                int from = randomAccount(random, lowerBound,
upperBound);
                int to = randomAccount(random, lowerBound, upperBound);
                while (to == from) {
                    to = randomAccount(random, lowerBound, upperBound);
                }

                int amount = 1 +
random.nextInt(config.maxTransferAmount());
                operations.add(new TransferOperation(from, to, amount));
            }

            return operations;
        }

        private static int randomAccount(Random random, int lowerBound,
int upperBound) {
            return lowerBound + random.nextInt(upperBound - lowerBound +
1);
        }
    }

    public static final class Database {
        private final String jdbcUrl;

        public Database(String jdbcUrl) {
```

```
            this.jdbcUrl = jdbcUrl;
        }

        public void initializeSchema() throws SQLException {
            try (
                    Connection connection = openConnection();
                    Statement statement = connection.createStatement()
            ) {
                statement.execute("""
                        CREATE TABLE IF NOT EXISTS accounts (
                            id INT PRIMARY KEY,
                            balance INT NOT NULL
                        )
                        """);
                statement.execute("""
                        CREATE TABLE IF NOT EXISTS transfer_log (
                            id BIGINT AUTO_INCREMENT PRIMARY KEY,
                            from_account INT NOT NULL,
                            to_account INT NOT NULL,
                            amount INT NOT NULL,
                            created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP NOT NULL
                        )
                        """);
                connection.commit();
            }
        }

        public void resetData(int accountCount, int initialBalance)
throws SQLException {
            try (
                    Connection connection = openConnection();
                    Statement statement = connection.createStatement();
                    PreparedStatement insert =
connection.prepareStatement(
                            "INSERT INTO accounts(id, balance) VALUES(?,
?)"
                    )
            ) {
                statement.executeUpdate("DELETE FROM transfer_log");
                statement.executeUpdate("DELETE FROM accounts");
                for (int accountId = 1; accountId <= accountCount;
accountId++) {
                    insert.setInt(1, accountId);
                    insert.setInt(2, initialBalance);
                    insert.addBatch();
```

```
                }
                insert.executeBatch();
                connection.commit();
            }
        }

        public Connection openConnection() throws SQLException {
            Connection connection =
DriverManager.getConnection(jdbcUrl);
            connection.setAutoCommit(false);

connection.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED
);
            return connection;
        }
    }

    public static final class TransferExecutor {
        public TransferExecutionOutcome execute(Connection connection,
TransferOperation operation) throws SQLException {
            int firstId = Math.min(operation.fromAccount(),
operation.toAccount());
            int secondId = Math.max(operation.fromAccount(),
operation.toAccount());

            /* Lock in order, so we don't get deadlock
             * 3 -> 7 => locked 3
             * 7 -> 3 => locked 7
             * 3 -> 7 => try to lock 7, but 7 is locked by 7 -> 3, so
wait
             */
            int firstBalance = readBalanceForUpdate(connection,
firstId);
            int secondBalance = readBalanceForUpdate(connection,
secondId);

            int fromBalance = operation.fromAccount() == firstId ?
firstBalance : secondBalance;
            if (fromBalance < operation.amount()) {
                connection.rollback();
                return TransferExecutionOutcome.INSUFFICIENT_FUNDS;
            }

            updateBalance(connection, operation.fromAccount(),
-operation.amount());
            updateBalance(connection, operation.toAccount(),
```

```
operation.amount());
            insertTransferLog(connection, operation);
            connection.commit();
            return TransferExecutionOutcome.SUCCESS;
        }

        private int readBalanceForUpdate(Connection connection, int
accountId) throws SQLException {
            try (
                    PreparedStatement statement =
connection.prepareStatement(
                            // FOR UPDATE to lock the row for the
duration of the transaction
                            "SELECT balance FROM accounts WHERE id = ?
FOR UPDATE"
                    )
            ) {
                statement.setInt(1, accountId);
                try (ResultSet resultSet = statement.executeQuery()) {
                    if (!resultSet.next()) {
                        throw new SQLException("Account not found: " +
accountId);
                    }
                    return resultSet.getInt(1);
                }
            }
        }

        private void updateBalance(Connection connection, int accountId,
int delta) throws SQLException {
            try (
                    PreparedStatement statement =
connection.prepareStatement(
                            "UPDATE accounts SET balance = balance + ?
WHERE id = ?"
                    )
            ) {
                statement.setInt(1, delta);
                statement.setInt(2, accountId);
                statement.executeUpdate();
            }
        }

        private void insertTransferLog(Connection connection,
TransferOperation operation) throws SQLException {
            try (
```

```
                    PreparedStatement statement =
connection.prepareStatement(
                            "INSERT INTO transfer_log(from_account,
to_account, amount) VALUES(?, ?, ?)"
                    )
            ) {
                statement.setInt(1, operation.fromAccount());
                statement.setInt(2, operation.toAccount());
                statement.setInt(3, operation.amount());
                statement.executeUpdate();
            }
        }
    }

    public static final class BenchmarkRunner {
        private final Database database;

        public BenchmarkRunner(Database database) {
            this.database = database;
        }

        public List<BenchmarkResult> run(BenchmarkConfig config) throws
SQLException, ExecutionException, InterruptedException {
            database.initializeSchema();

            var workload = WorkloadGenerator.generate(config);
            var results = new ArrayList<BenchmarkResult>();

            for (int threadCount : config.threads()) {
                for (ExecutionMode mode : config.modes()) {
                    for (int repeat = 1; repeat <= config.repeats();
repeat++) {
                        database.resetData(config.accountCount(),
config.initialBalance());
                        CoordinationStrategy coordination =
createCoordination(mode, config.accountCount());
                        BenchmarkResult result = runSingle(workload,
threadCount, mode, repeat, coordination);
                        results.add(result);
                    }
                }
            }

            return results;
        }
```

```java
        private BenchmarkResult runSingle(
                List<TransferOperation> workload,
                int threadCount,
                ExecutionMode mode,
                int repeat,
                CoordinationStrategy coordination
        ) throws InterruptedException, ExecutionException {
            ExecutorService executor =
Executors.newFixedThreadPool(threadCount);
            AtomicInteger nextIndex = new AtomicInteger(0);
            TransferExecutor transferExecutor = new TransferExecutor();
            long start = System.nanoTime();

            try {
                List<Future<WorkerMetrics>> futures = new ArrayList<>();
                for (int i = 0; i < threadCount; i++) {
                    futures.add(executor.submit(createWorker(workload,
nextIndex, coordination, transferExecutor)));
                }

                WorkerMetrics metrics = new WorkerMetrics(0, 0, 0, 0);
                for (Future<WorkerMetrics> future : futures) {
                    metrics = metrics.add(future.get());
                }

                long elapsedNanos = System.nanoTime() - start;
                long elapsedMillis =
TimeUnit.NANOSECONDS.toMillis(elapsedNanos);
                double throughput = metrics.successfulTransfers() *
1_000_000_000.0 / elapsedNanos;
                double avgLatencyMicros = workload.isEmpty() ? 0.0 :
metrics.totalLatencyNanos() / 1_000.0 / workload.size();
                return new BenchmarkResult(
                        mode,
                        threadCount,
                        repeat,
                        elapsedMillis,
                        metrics.successfulTransfers(),
                        metrics.insufficientFunds(),
                        metrics.sqlErrors(),
                        throughput,
                        avgLatencyMicros
                );
            } finally {
                executor.shutdown();
                executor.awaitTermination(1, TimeUnit.MINUTES);
```

```
            }
        }

        private Callable<WorkerMetrics> createWorker(
                List<TransferOperation> workload,
                AtomicInteger nextIndex,
                CoordinationStrategy coordination,
                TransferExecutor transferExecutor
        ) {
            return () -> {
                long successfulTransfers = 0;
                long insufficientFunds = 0;
                long sqlErrors = 0;
                long totalLatencyNanos = 0;

                try (Connection connection = database.openConnection())
{
                    while (true) {
                        int currentIndex = nextIndex.getAndIncrement();
                        if (currentIndex >= workload.size()) {
                            break;
                        }

                        TransferOperation operation =
workload.get(currentIndex);
                        long started = System.nanoTime();

coordination.beforeTransfer(operation.fromAccount(),
operation.toAccount());
                        try {
                            TransferExecutionOutcome outcome =
transferExecutor.execute(connection, operation);
                            if (outcome ==
TransferExecutionOutcome.SUCCESS) {
                                successfulTransfers++;
                            } else {
                                insufficientFunds++;
                            }
                        } catch (SQLException e) {
                            sqlErrors++;
                            connection.rollback();
                        } finally {

coordination.afterTransfer(operation.fromAccount(),
operation.toAccount());
                            totalLatencyNanos += System.nanoTime() -
```

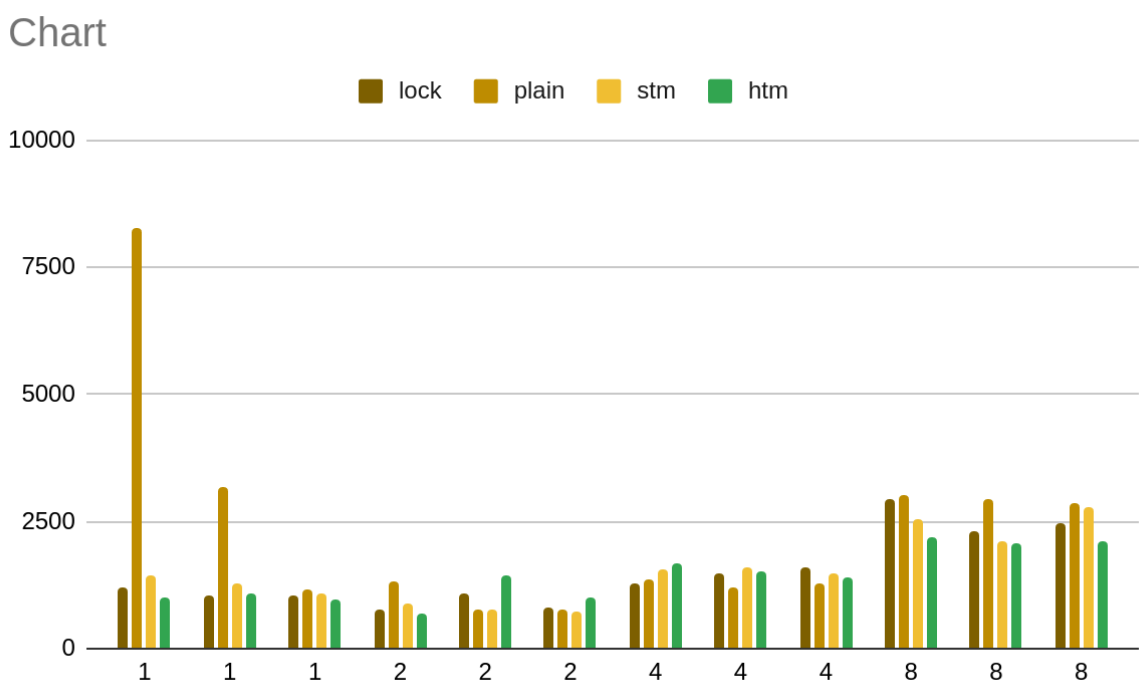
```
started;
                        }
                    }
                }

                return new WorkerMetrics(successfulTransfers,
insufficientFunds, sqlErrors, totalLatencyNanos);
            };
        }

        private CoordinationStrategy createCoordination(ExecutionMode
mode, int accountCount) {
            return switch (mode) {
                case PLAIN -> new NoOpCoordinationStrategy();
                case LOCK -> new
SynchronizedCoordinationStrategy(accountCount);
                case STM -> new StmCoordinationStrategy(accountCount);
            };
        }
    }

    public static final class ResultPrinter {
        private ResultPrinter() {
        }

        public static void print(List<BenchmarkResult> results) {
            System.out.printf("%-6s %-7s %-6s %-10s %-10s %-10s %-10s
%-14s %-14s%n",
                    "mode", "threads", "run", "time_ms", "success",
"no_funds", "sql_err", "ops_per_sec", "avg_us");

            results.stream()
                    .sorted(Comparator.comparing(BenchmarkResult::mode)
                            .thenComparingInt(BenchmarkResult::threads)
                            .thenComparingInt(BenchmarkResult::repeat))
                    .forEach(result -> System.out.printf(
                            "%-6s %-7d %-6d %-10d %-10d %-10d %-10d
%-14.2f %-14.2f%n",
                            result.mode().name().toLowerCase(),
                            result.threads(),
                            result.repeat(),
                            result.elapsedMillis(),
                            result.successfulTransfers(),
                            result.insufficientFunds(),
                            result.sqlErrors(),
                            result.throughputOpsPerSecond(),
```

```
                                result.averageLatencyMicros()
                        ));
        }
    }

    public static final class CsvResultWriter {
        private CsvResultWriter() {
        }

        public static void write(Path output, List<BenchmarkResult>
results) throws IOException {
            Path absoluteOutput = output.toAbsolutePath();
            Path parent = absoluteOutput.getParent();
            if (parent != null) {
                Files.createDirectories(parent);
            }

            StringBuilder csv = new StringBuilder();

csv.append("mode,threads,repeat,elapsed_ms,successful_transfers,insuffic
ient_funds,sql_errors,throughput_ops_per_sec,average_latency_micros\n");
            for (BenchmarkResult result : results) {

csv.append(result.mode().name().toLowerCase()).append(',')
                        .append(result.threads()).append(',')
                        .append(result.repeat()).append(',')
                        .append(result.elapsedMillis()).append(',')

.append(result.successfulTransfers()).append(',')
                        .append(result.insufficientFunds()).append(',')
                        .append(result.sqlErrors()).append(',')

.append(result.throughputOpsPerSecond()).append(',')

.append(result.averageLatencyMicros()).append('\n');
            }

            Files.writeString(output, csv);
        }
    }
}

final class NoOpCoordinationStrategy implements CoordinationStrategy {
    @Override
    public void beforeTransfer(int fromAccount, int toAccount) {
    }
```

```java
    @Override
    public void afterTransfer(int fromAccount, int toAccount) {
    }
}

final class SynchronizedCoordinationStrategy implements
CoordinationStrategy {
    private final boolean[] busyAccounts;

    public SynchronizedCoordinationStrategy(int accountCount) {
        this.busyAccounts = new boolean[accountCount + 1];
    }

    @Override
    public void beforeTransfer(int fromAccount, int toAccount) {
        synchronized (busyAccounts) {
            while (busyAccounts[fromAccount] || busyAccounts[toAccount])
{
                try {
                    busyAccounts.wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    throw new IllegalStateException("Interrupted while
waiting for account reservation", e);
                }
            }
            busyAccounts[fromAccount] = true;
            busyAccounts[toAccount] = true;
        }
    }

    @Override
    public void afterTransfer(int fromAccount, int toAccount) {
        synchronized (busyAccounts) {
            busyAccounts[fromAccount] = false;
            busyAccounts[toAccount] = false;
            busyAccounts.notifyAll();
        }
    }
}

final class StmCoordinationStrategy implements CoordinationStrategy {
    private final TxnInteger[] busyAccounts;

    public StmCoordinationStrategy(int accountCount) {
```

```
        this.busyAccounts = new TxnInteger[accountCount + 1];
        for (int accountId = 1; accountId <= accountCount; accountId++)
{
            busyAccounts[accountId] = newTxnInteger(0);
        }
    }

    @Override
    public void beforeTransfer(int fromAccount, int toAccount) {
        atomic(() -> {
            if (busyAccounts[fromAccount].get() > 0 ||
busyAccounts[toAccount].get() > 0) {
                retry();
            }
            busyAccounts[fromAccount].increment();
            busyAccounts[toAccount].increment();
        });
    }

    @Override
    public void afterTransfer(int fromAccount, int toAccount) {
        atomic(() -> {
            busyAccounts[fromAccount].decrement();
            busyAccounts[toAccount].decrement();
        });
    }
}
```

Графіки



mode	threads	repeat	elapsed_ms	successful_transfers	insufficient_funds
plain	1	1	8283	49832	168
plain	1	2	3189	49832	168
plain	1	3	1182	49832	168
lock	1	1	1193	49832	168
lock	1	2	1028	49832	168
lock	1	3	1031	49832	168
stm	1	1	1443	49832	168
stm	1	2	1278	49832	168
stm	1	3	1082	49832	168
plain	2	1	1332	49832	168
plain	2	2	750	49832	168
plain	2	3	753	49836	164
lock	2	1	788	49834	166
lock	2	2	1067	49826	174
lock	2	3	797	49826	174
stm	2	1	898	49826	174
stm	2	2	767	49833	167
stm	2	3	740	49834	166
plain	4	1	1349	49834	166

plain	4	2	1184	49833	167
plain	4	3	1282	49838	162
lock	4	1	1285	49834	166
lock	4	2	1467	49828	172
lock	4	3	1607	49835	165
stm	4	1	1564	49825	175
stm	4	2	1604	49834	166
stm	4	3	1471	49836	164
plain	8	1	3032	49833	167
plain	8	2	2928	49837	163
plain	8	3	2877	49837	163
lock	8	1	2945	49845	155
lock	8	2	2313	49836	164
lock	8	3	2457	49836	164
stm	8	1	2538	49830	170
stm	8	2	2098	49837	163
stm	8	3	2775	49833	167
mode	threads	repeat	elapsed_ms	successful_transfers	insufficient_funds
htm	1	1	999	49832	168
htm	1	2	1075	49832	168
htm	1	3	985	49832	168
htm	2	1	686	49826	174
htm	2	2	1455	49833	167
htm	2	3	1017	49834	166
htm	4	1	1665	49825	175
htm	4	2	1511	49834	166
htm	4	3	1398	49836	164
htm	8	1	2182	49830	170
htm	8	2	2064	49837	163
htm	8	3	2123	49833	167

Висновок

У ході роботи було реалізовано багатопотокову Java-програму для моделювання банківських переказів через JDBC API. Як сховище використано базу даних, а кожна операція переказу виконувалась як окрема SQL-транзакція з блокуванням рядків через SELECT ... FOR UPDATE, подальшими UPDATE та фіксацією змін. Для дослідження було реалізовано чотири режими виконання: plain без додаткової синхронізації на рівні Java, lock із явним блокуванням, stm із software transactional memory та htm у середовищі, де база даних і програма запускались на одному комп'ютері.

За отриманими результатами не спостерігається чіткої та сталої переваги одного підходу над іншими. Хоча htm показав найкращий середній час при 1 потоці (приблизно 1020 мс), він не виявився постійно найшвидшим: були запуски де інші варіанти запуску, наприклад plain - 9-тий запуск із 4-ма потоками, були швидші. Це свідчить про те, що для даного навантаження сам факт використання HTM не гарантує стабільного прискорення. Усі режими демонструють близький порядок часу виконання, а різниця між ними є помірною та залежить від конкретного запуску.

Причина цього може бути в тому, що основне навантаження припадає не на механізм синхронізації в Java, а на роботу самої СУБД. У всіх режимах виконується однакова SQL-транзакція, яка включає блокування рядків, читання, оновлення та журналювання. Тому вирішальним фактором є не стільки спосіб координації потоків у Java, скільки конфліктний доступ до одних і тих самих записів у базі даних. Навіть при ввімкненому HTM не помітний значний виграш, який може нівелюватися тим, що з 50 000 операцій на запуск, програма мала 80% на 20 "гарячих" рахунків із 1000, що створювало значну конкуренцію за одні й ті самі записи.

Також слід відзначити аномально великі значення часу для перших двох запусків режиму plain при 1-му потоці: 8283 мс і 3189 мс, тоді як третій запуск дав уже 1182 мс. Найімовірніше, це пояснюється ефектом прогріву JVM, JIT-компіляції, ініціалізації пулів, потоків, JDBC-компонентів і внутрішніх структур бази даних. Саме тому ці перші результати не варто розглядати як характерні для сталого режиму роботи. Після прогріву значення plain стає співмірним з іншими методами.

Додатково можна помітити, що кількість успішних переказів та кількість операцій з недостатністю коштів в усіх режимах дуже близькі, але неоднакові в залежності від режиму та кількості тредів. Це означає, що всі підходи забезпечили подібну функціональну коректність, а основна відмінність між ними проявилась лише в часових характеристиках.

Отже, за результатами експерименту можна зробити висновок, що в даній реалізації HTM не продемонструвала безумовної переваги над HTM, хоча в окремих конфігураціях, вона дала найкращий результат. Загалом усі підходи виявилися близькими за продуктивністю. Таким чином, у цій роботі доцільно говорити не про явне домінування HTM, а про те, що її ефективність суттєво залежить від характеру навантаження і може бути обмежена накладними витратами.